

Modul Praktikum Flutter Lanjutan

Pertemuan 9: REST API CRUD dengan FastAPI, SQLAlchemy, dan Alembic

Durasi	Level	Bentuk kerja	Prasyarat
150 menit	Lanjut	Individu	Pertemuan 8 (stack Docker Compose berjalan)

1. Tujuan Pembelajaran

Setelah praktikum ini, mahasiswa mampu:

1. Merancang endpoint REST untuk satu sumber daya: method, URL, kode status, dan format JSON.
2. Memetakan tabel ke kelas Python dengan ORM **SQLAlchemy 2**.
3. Mengelola perubahan skema database dengan migrasi **Alembic**.
4. Memvalidasi data masuk dan aturan bisnis dengan **Pydantic**.
5. Membuat endpoint CRUD lengkap dengan filter, pagination, `PUT`, dan `PATCH`, lalu mengujinya di Swagger UI.
6. Membuat lapisan layanan API di Flutter yang dapat diuji tanpa server.

2. Alat dan Bahan

- Hasil pertemuan 8 (atau snapshot `pert8/tugas8`).
- Browser untuk Swagger UI (<http://localhost:8000/docs>) dan Adminer (<http://localhost:8081>).

3. Teori Singkat

REST memodelkan data sebagai *resource* yang diakses lewat URL. Method HTTP menentukan aksinya:

Method + URL	Aksi	Kode sukses	Kode gagal umum
GET /kegiatan?tanggal=...	Daftar (dengan filter)	200	422 parameter salah
GET /kegiatan/{id}	Detail satu data	200	404 tidak ditemukan
POST /kegiatan	Membuat data baru	201 Created	422 data tidak valid
PUT /kegiatan/{id}	Mengganti seluruh data	200	404, 422
PATCH /kegiatan/{id}	Mengubah sebagian field	200	404, 422
DELETE /kegiatan/{id}	Menghapus	204 No Content	404

ORM (Object Relational Mapping) memetakan tabel menjadi kelas, sehingga kode cukup memanggil `db.add(k)` tanpa menulis `INSERT` secara manual. Query yang dibuat ORM memakai parameter, jadi aman dari SQL injection. SQLAlchemy 2 memakai anotasi `Mapped[...]` sebagai sumber tipe kolom.

Migrasi. Pada pertemuan 7 skema dibuat di `onCreate` dan diubah lewat `onUpgrade` dengan `version`. Alembic menerapkan konsep yang sama di server: setiap perubahan skema ditulis sebagai berkas *revisi* bernomor (0001, 0002, ...). Perintah `alembic upgrade head` menerapkan revisi yang belum dijalankan, dan riwayatnya dicatat di tabel `alembic_version`. Dengan cara ini, semua laptop dan server memiliki skema yang sama.

Pydantic memeriksa JSON yang masuk berdasarkan *type hint*. Bila ada yang salah, FastAPI otomatis membalas **422** beserta lokasi dan pesan galatnya. Aturan bisnis pertemuan 7 (nominal > 0, kategori wajib untuk pengeluaran) ditulis sebagai *validator*.

Session per request. Setiap request mendapat satu `Session` database dari *dependency* `get_db`. `Session` itu otomatis ditutup setelah respons dikirim, termasuk saat terjadi galat.

SQLite (pert. 7)	MySQL (pert. 9)	Alasan
tanggal TEXT 'yyyy-MM-dd'	DATE	Tipe tanggal asli; bisa dibandingkan dan memakai BETWEEN
selesai INTEGER 0/1	BOOLEAN(tinyint(1))	Di JSON tetap true/false
tipe TEXT	ENUM('tanpa', 'pemasukan', 'pengeluaran')	Database menolak nilai di luar daftar
tidak ada	created_at, updated_at	Jejak waktu untuk admin dan sinkronisasi

4. Langkah Praktikum

Bagian A: Session dan Base

Tambahkan `alembic==1.20.0` ke `requirements.txt`. Lengkapi `backend/app/database.py`:

```
from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, sessionmaker

from .config import settings

# pool_pre_ping: cek koneksi sebelum dipakai, berguna bila MySQL sempat restart.
engine = create_engine(settings.database_url, pool_pre_ping=True)
SessionLocal = sessionmaker(bind=engine, autoflush=False, expire_on_commit=False)

class Base(DeclarativeBase):
    """Kelas induk semua model tabel."""

def get_db():
    """Dependency FastAPI: satu session per request, selalu ditutup."""
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Bagian B: Model Kegiatan

`backend/app/models.py`:

```
import enum
from datetime import date, datetime

from sqlalchemy import Date, DateTime, Enum, String, func
from sqlalchemy.orm import Mapped, mapped_column

from .database import Base

class Tipe(str, enum.Enum):
    tanpa = "tanpa"
    pemasukan = "pemasukan"
    pengeluaran = "pengeluaran"
```

```

class Kegiatan(Base):
    __tablename__ = "kegiatan"

    id: Mapped[int] = mapped_column(primary_key=True)
    judul: Mapped[str] = mapped_column(String(100))
    tanggal: Mapped[date] = mapped_column(Date, index=True)
    selesai: Mapped[bool] = mapped_column(default=False)
    tipe: Mapped[Tipe] = mapped_column(Enum(Tipe), default=Tipe.tanpa)
    nominal: Mapped[int] = mapped_column(default=0)
    kategori: Mapped[str] = mapped_column(String(30), default="-")
    created_at: Mapped[datetime] = mapped_column(DateTime, server_default=func.now())
    updated_at: Mapped[datetime] = mapped_column(
        DateTime, server_default=func.now(), onupdate=func.now()
    )

```

Kolom tanggal diberi `index=True` karena hampir setiap query memfilter berdasarkan tanggal.

Bagian C: Migrasi dengan Alembic

1. Buat `backend/alembic.ini`. Bagian penting hanya tiga baris berikut, sisanya konfigurasi log (salin dari snapshot pert9/praktikum9):

```

[alembic]
script_location = alembic
file_template = %% (rev)s_%% (slug)s
prepend_sys_path = .

```

2. Buat `backend/alembic/env.py`. URL database **tidak** ditulis di `alembic.ini`, tetapi dibaca dari konfigurasi aplikasi agar hanya ada satu sumber:

```

from logging.config import fileConfig

from alembic import context
from sqlalchemy import create_engine

from app import models  # noqa: F401 (mendaftarkan semua tabel ke Base.metadata)
from app.config import settings
from app.database import Base

if context.config.config_file_name is not None:
    fileConfig(context.config.config_file_name)

target_metadata = Base.metadata

def run_migrations_online() -> None:
    engine = create_engine(settings.database_url)
    with engine.connect() as connection:
        context.configure(connection=connection, target_metadata=target_metadata)
        with context.begin_transaction():
            context.run_migrations()

run_migrations_online()

```

Snapshot juga menyediakan mode *offline* (`alembic upgrade head --sql` untuk mencetak SQL tanpa menjalankannya) dan `alembic/script.py.mako` (templat berkas revisi).

3. Tulis revisi pertama `backend/alembic/versions/0001_buat_tabel_kegiatan.py`:

```

revision = "0001"
down_revision = None

def upgrade() -> None:
    op.create_table(
        "kegiatan",
        sa.Column("id", sa.Integer(), primary_key=True),
        sa.Column("judul", sa.String(100), nullable=False),

```

```

sa.Column("tanggal", sa.Date(), nullable=False),
sa.Column("selesai", sa.Boolean(), nullable=False, server_default=sa.false()),
sa.Column(
    "tipe",
    sa.Enum("tanpa", "pemasukan", "pengeluaran", name="tipe"),
    nullable=False,
    server_default="tanpa",
),
sa.Column("nominal", sa.Integer(), nullable=False, server_default="0"),
sa.Column("kategori", sa.String(30), nullable=False, server_default="-"),
sa.Column("created_at", sa.DateTime(), nullable=False, server_default=sa.func.now()),
sa.Column("updated_at", sa.DateTime(), nullable=False, server_default=sa.func.now()),
)
op.create_index("ix_kegiatan_tanggal", "kegiatan", ["tanggal"])

def downgrade() -> None:
    op.drop_index("ix_kegiatan_tanggal", table_name="kegiatan")
    op.drop_table("kegiatan")

```

Autogenerate. Alembic dapat membuat revisi otomatis dengan membandingkan model dan database:

`docker compose exec api alembic revision --autogenerate -m "pesan".` Hasilnya **wajib diperiksa** sebelum dipakai, karena perubahan seperti mengganti nama kolom terbaca sebagai hapus lalu tambah (data hilang). Pada modul ini revisi ditulis manual dengan nomor tetap agar sama di semua snapshot.

4. Migrasi dijalankan otomatis setiap kali container API menyala. Buat `backend/start.sh`:

```

#!/bin/sh
set -e

echo "Menjalankan migrasi database..."
alembic upgrade head

echo "Menjalankan API..."
exec uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload

```

Ganti baris terakhir Dockerfile menjadi `CMD ["sh", "start.sh"]`. Di `docker-compose.yml`, ubah *bind mount* API menjadi seluruh folder backend agar berkas revisi hasil *autogenerate* muncul di laptop:

```

volumes:
  - ./backend:/code

```

5. Jalankan `docker compose up -d --build`, lalu periksa:

```

docker compose logs api | grep alembic      # Running upgrade -> 0001
docker compose exec api alembic current    # 0001 (head)
docker compose exec api alembic check      # No new upgrade operations detected.

```

Checkpoint C: di Adminer terlihat tabel kegiatan dan `alembic_version`. `alembic check` tidak menemukan perbedaan antara model dan database.

Bagian D: Skema Pydantic dan Aturan Bisnis

`backend/app/schemas.py` memisahkan data **masuk** (`KegiatanInput`) dan data **keluar** (`KegiatanOut`):

```

KATEGORI_PENGELUARAN = ["Makan", "Transport", "Belajar", "Hiburan", "Lainnya"]

class KegiatanBase(BaseModel):
    judul: str = Field(min_length=1, max_length=100, examples=["Makan siang"])

```

```

tanggal: date = Field(examples=["2026-10-06"])
selesai: bool = False
tipe: Tipe = Tipe.tanpa
nominal: int = Field(default=0, ge=0, examples=[15000])
kategori: str = Field(default="-", examples=["Makan"])

class KegiatanInput(KegiatanBase):
    """Data dari klien untuk POST dan PUT. Aturan bisnis modul 7 dicek di sini."""

    @field_validator("judul")
    @classmethod
    def judul_tidak_kosong(cls, v: str) -> str:
        v = v.strip()
        if not v:
            raise ValueError("judul wajib diisi")
        return v

    @model_validator(mode="after")
    def cek_aturan_bisnis(self):
        if self.tipe == Tipe.tanpa:
            self.nominal = 0
            self.kategori = "-"
            return self
        if self.nominal <= 0:
            raise ValueError("nominal wajib lebih dari 0 untuk pemasukan/pengeluaran")
        if self.tipe == Tipe.pemasukan:
            self.kategori = "-"
        elif self.kategori not in KATEGORI_PENGELUARAN:
            raise ValueError(
                "kategori pengeluaran harus salah satu dari: " + ", ".join(KATEGORI_PENGELUARAN)
            )
        return self

class KegiatanPatch(BaseModel):
    """PATCH: semua field opsional, hanya yang dikirim yang diubah."""

    judul: str | None = None
    tanggal: date | None = None
    selesai: bool | None = None
    tipe: Tipe | None = None
    nominal: int | None = None
    kategori: str | None = None

class KegiatanOut(KegiatanBase):
    model_config = ConfigDict(from_attributes=True)

    id: int
    created_at: datetime
    updated_at: datetime

```

`from_attributes=True` memungkinkan `KegiatanOut` dibuat langsung dari objek SQLAlchemy.

Bagian E: Router CRUD

Pindahkan endpoint `/health` ke `app/routers/health.py` memakai

`APIRouter(tags=["health"])`. Lalu buat `app/routers/kegiatan.py`. Potongan terpenting:

```

router = APIRouter(prefix="/kegiatan", tags=["kegiatan"])

def _ambil_atau_404(db: Session, kegiatan_id: int) -> Kegiatan:
    k = db.get(Kegiatan, kegiatan_id)
    if k is None:
        raise HTTPException(status.HTTP_404_NOT_FOUND, "Kegiatan tidak ditemukan")
    return k

@router.get("", response_model=list[KegiatanOut])
def daftar_kegiatan(

```

```

response: Response,
tanggal: date | None = None,
tipe: Tipe | None = None,
q: str | None = Query(None, description="Cari di judul"),
limit: int = Query(50, ge=1, le=200),
offset: int = Query(0, ge=0),
db: Session = Depends(get_db),
):
    stmt = select(Kegiatan)
    if tanggal:
        stmt = stmt.where(Kegiatan.tanggal == tanggal)
    if tipe:
        stmt = stmt.where(Kegiatan.tipe == tipe)
    if q:
        stmt = stmt.where(Kegiatan.judul.contains(q))

    total = db.scalar(select(func.count()).select_from(stmt.subquery()))
    response.headers["X-Total-Count"] = str(total)

    stmt = stmt.order_by(Kegiatan.selesai, Kegiatan.id.desc()).limit(limit).offset(offset)
    return db.scalars(stmt).all()

@router.post("", response_model=KegiatanOut, status_code=status.HTTP_201_CREATED)
def tambah_kegiatan(data: KegiatanInput, db: Session = Depends(get_db)):
    k = Kegiatan(**data.model_dump())
    db.add(k)
    db.commit()
    db.refresh(k) # baca ulang: id dan created_at diisi oleh database
    return k

```

PUT memakai pola yang sama dengan setattr untuk setiap field. PATCH perlu kehati-hatian: field yang dikirim **digabung** dengan data lama, lalu **divalidasi ulang**. Tanpa langkah ini, {"nominal": 0} pada pengeluaran akan lolos dan melanggar aturan bisnis:

```

@router.patch("/{kegiatan_id}", response_model=KegiatanOut)
def ubah_sebagian(kegiatan_id: int, data: KegiatanPatch, db: Session = Depends(get_db)):
    """Contoh: {"selesai": true} untuk mencentang kegiatan."""
    k = _ambil_atau_404(db, kegiatan_id)
    gabungan = KegiatanOut.model_validate(k).model_dump()
    gabungan.update(data.model_dump(exclude_unset=True))
    try:
        # Validasi ulang agar aturan bisnis tetap berlaku setelah digabung.
        valid = KegiatanInput.model_validate(gabungan)
    except ValidationError as e:
        # Dibalas 422 dengan format yang sama seperti validasi bawaan FastAPI.
        raise RequestValidationError(e.errors(include_url=False, include_context=False)) from e
    for kolom, nilai in valid.model_dump().items():
        setattr(k, kolom, nilai)
    db.commit()
    db.refresh(k)
    return k

@router.delete("/{kegiatan_id}", status_code=status.HTTP_204_NO_CONTENT)
def hapus_kegiatan(kegiatan_id: int, db: Session = Depends(get_db)):
    db.delete(_ambil_atau_404(db, kegiatan_id))
    db.commit()

```

Daftarkan router di main.py: `app.include_router(health.router)` dan `app.include_router(kegiatan.router)`.

Bagian F: Menguji di Swagger UI

Buka /docs, pilih endpoint, klik **Try it out**, lalu **Execute**. Isi tabel berikut di laporan:

No	Permintaan	Hasil yang diharapkan
1	POST pengeluaran Rp 15.000 kategori Makan	201; respons berisi id, created_at

No	Permintaan	Hasil yang diharapkan
2	POST pemasukan dengan kategori: "Makan"	201; kategori otomatis menjadi "-"
3	POST dengan judul: " "	422 judul wajib diisi
4	POST pengeluaran kategori: "Jajan"	422 dengan daftar kategori yang sah
5	PATCH /kegiatan/1 {"selesai": true}	200; hanya selesai dan updated_at berubah
6	PATCH /kegiatan/1 {"nominal": 0}	422; data tidak berubah
7	GET /kegiatan?tanggal=2026-10-06&limit=1	Satu data; header x-total-count berisi jumlah seluruhnya
8	DELETE /kegiatan/2, lalu GET /kegiatan/2	204, lalu 404

Checkpoint F: semua skenario sesuai, dan data di Adminer sama dengan hasil GET.

Bagian G: Lapisan API di Flutter

Pada pertemuan ini UI belum diubah (masih SQLite). Yang dibuat adalah **lapisan layanan** yang akan dipakai UI di pertemuan 11.

1. Tambahkan konversi JSON di model Kegiatan. Bedanya dengan `toMap()` SQLite: selesai berupa boolean dan id tidak dikirim karena sudah ada di URL:

```
Map<String, Object?> toJson() => {
  'judul': judul,
  'tanggal': tanggal,
  'selesai': selesai,
  'tipe': tipe.name,
  'nominal': nominal,
  'kategori': kategori,
};

factory Kegiatan.fromJson(Map<String, dynamic> j) => Kegiatan(
  id: j['id'] as int,
  judul: j['judul'] as String,
  tanggal: j['tanggal'] as String,
  selesai: j['selesai'] as bool,
  tipe: Tipe.values.byName(j['tipe'] as String),
  nominal: j['nominal'] as int,
  kategori: j['kategori'] as String,
);
```

2. Buat `lib/services/api_exception.dart` yang menerjemahkan dua bentuk galat FastAPI menjadi pesan siap tampil:

```
class ApiException implements Exception {
  final int? kode;
  final String pesan;

  const ApiException(this.pesan, {this.kode});

  /// Membaca field `detail` dari respons FastAPI:
  /// - HTTPException -> {"detail": "Kegiatan tidak ditemukan"}
  /// - validasi (422) -> {"detail": [{"loc": [...], "msg": "..."}]}
  factory ApiException.dariRespons(http.Response res) {
    String pesan = 'Server membalas kode ${res.statusCode}';
    try {
      final detail = (jsonDecode(res.body) as Map<String, dynamic>)['detail'];
      if (detail is String) {
        pesan = detail;
      }
    }
  }
}
```

```

    } else if (detail is List) {
      pesan = detail
        .map((e) => (e['msg'] as String).replaceFirst('Value error, ', ''))
        .join('\n');
    }
  } catch (_) {
    // Badan respons bukan JSON (misalnya halaman galat 502 dari proxy).
  }
  return ApiException(pesan, kode: res.statusCode);
}

@override
String toString() => pesan;
}

```

3. Buat `lib/services/kegiatan_api.dart` berisi kelas `KegiatanApi` dengan method `daftar({tanggal})`, `tambah`, `ubah (PUT)`, `setSelesai (PATCH)`, dan `hapus`. Setiap respons di luar 2xx dilempar sebagai `ApiException`:

```

class KegiatanApi {
  final http.Client _client;

  KegiatanApi({http.Client? client}) : _client = client ?? http.Client();

  static const _json = {'Content-Type': 'application/json'};

  Uri _uri(String path, [Map<String, String>? query]) =>
    Uri.parse('${apiUrl.value}$path').replace(queryParameters: query);

  http.Response _cek(http.Response res) {
    if (res.statusCode < 200 || res.statusCode >= 300) {
      throw ApiException.dariRespons(res);
    }
    return res;
  }

  Future<List<Kegiatan>> daftar({String? tanggal}) async {
    final res = _cek(
      await _client
        .get(_uri('/kegiatan', {'tanggal': ?tanggal}))
        .timeout(batasWaktu),
    );
    final data = jsonDecode(res.body) as List<dynamic>;
    return data.map((e) => Kegiatan.fromJson(e as Map<String, dynamic>)).toList();
  }

  Future<Kegiatan> setSelesai(int id, bool selesai) async {
    final res = _cek(
      await _client
        .patch(
          _uri('/kegiatan/$id'),
          headers: _json,
          body: jsonEncode({'selesai': selesai}),
        )
        .timeout(batasWaktu),
    );
    return Kegiatan.fromJson(jsonDecode(res.body) as Map<String, dynamic>);
  }

  // tambah (POST), ubah (PUT), hapus (DELETE) mengikuti pola yang sama.
}

```

`{'tanggal': ?tanggal}` adalah *null-aware element* (Dart 3.8+): entri hanya dimasukkan bila tanggal tidak null. `apiUrl.value` berasal dari tugas pertemuan 8 (alamat API yang dapat diubah).

4. Uji dengan `MockClient` di `test/kegiatan_api_test.dart`, minimal: `fromJson/toJson`, filter tanggal pada URL, POST mengirim JSON, PATCH hanya mengirim selesai, dan pesan 422 terbaca dari detail.

Checkpoint G: flutter test lulus dan flutter analyze tanpa masalah.

5. Latihan Mandiri

1. Tambahkan filter `selesai` (boolean) pada `GET /kegiatan`.
2. Tambahkan parameter `urut` dengan nilai terbaru/terlama untuk mengganti urutan.
3. Buat revisi **Alembic 0001b** yang menambah kolom `catatan` (TEXT, nullable) memakai `--autogenerate`. Periksa isinya, jalankan, lalu `alembic downgrade -1` dan amati perubahan di Adminer. (Hapus revisi tersebut sebelum melanjutkan ke pertemuan 10.)
4. Apa yang terjadi bila mengirim `"tanggal": "06-10-2026"`? Jelaskan pesan galatnya.

6. Tugas

Lengkapi API dengan **endpoint laporan**, lalu buat pembungkusnya di Flutter:

- `GET /laporan/harian?tanggal=YYYY-MM-DD` **membalas** jumlah_kegiatan, selesai, pemasukan, pengeluaran, tabungan, tanggal, dan per_kategori (daftar {kategori, total} diurutkan dari terbesar). Hitung di **SQL** (COUNT, SUM(CASE WHEN ...), COALESCE, GROUP BY), bukan dengan perulangan Python atas seluruh data.
- `GET /laporan/rentang?mulai=...&sampai=...` **membalas** ringkasan **setiap hari** dalam rentang, termasuk hari tanpa kegiatan (bernilai 0). Gunakan satu query `GROUP BY tanggal`. Tolak rentang terbalik atau lebih dari 31 hari dengan **400**.
- Hasil untuk data "Contoh Skenario Satu Hari" modul 7 harus persis: 4 kegiatan, 2 selesai, pemasukan 50.000, pengeluaran 25.000 (Makan 15.000, Transport 10.000), tabungan 25.000.
- Flutter: Ringkasan.fromJson, kelas LaporanHarian, dan LaporanApi dengan method harian(tanggal) dan rentang(mulai, sampai), beserta unit test MockClient.
- Kumpulkan: kode, tangkapan layar Swagger untuk kedua endpoint, dan tabel hasil uji bagian F.

7. Rubrik Penilaian

Komponen	Bobot
Bagian A–G berjalan (checkpoint), migrasi Alembic benar	30%
Tabel uji Swagger (bagian F) lengkap	10%
Latihan mandiri	15%
Tugas: endpoint laporan benar dan dihitung di SQL	30%
Tugas: LaporanApi + unit test	15%

8. Pertanyaan Refleksi

1. Mengapa `POST` membalas 201 dan `DELETE` membalas 204, bukan 200?
2. Apa yang dapat terjadi bila skema database diubah langsung lewat Adminer, bukan lewat revisi Alembic?
3. Mengapa `PATCH` memvalidasi ulang data gabungan? Beri contoh data rusak yang dicegahnya.
4. Mengapa pagination penting untuk aplikasi mobile? Apa fungsi header `X-Total-Count`?
5. Apa keuntungan KegiatanApi menerima `http.Client` lewat konstruktor?

9. Troubleshooting Umum

Masalah	Solusi
Can't locate revision identified by '...'	Database berisi revisi yang tidak ada di folder <code>alembic/versions</code> (misalnya setelah menghapus berkas revisi). Saat pengembangan: <code>docker compose down -v</code> lalu jalankan ulang
Table 'kegiatan' already exists	Tabel dibuat manual di luar Alembic. Hapus tabel itu, atau tandai revisi dengan <code>alembic stamp 0001</code>
ModuleNotFoundError: No module named 'app' saat alembic	Jalankan dari folder <code>/code</code> (bawaan container). Pastikan <code>prepend_sys_path = .</code> ada di <code>alembic.ini</code>
422 padahal data terlihat benar	Baca <code>detail[].loc</code> di respons: menunjukkan field yang salah. Format tanggal harus <code>YYYY-MM-DD</code>
Object of type date is not JSON serializable (500)	Jangan mengirim objek Pydantic mentah ke <code>HTTPException</code> . Pakai <code>RequestValidationError</code> seperti contoh PATCH
Perubahan model tidak terdeteksi <code>--autogenerate</code>	Model harus diimpor di <code>alembic/env.py</code> (<code>from app import models</code>)
Flutter type 'Null' is not a subtype of type 'int'	Nama field JSON salah eja atau field opsional. Bandingkan dengan respons di Swagger

10. Referensi

- FastAPI, SQL (*Relational*) Databases: fastapi.tiangolo.com/tutorial/sql-databases
- SQLAlchemy 2.0 ORM Quick Start: docs.sqlalchemy.org/en/20/orm/quickstart.html
- Alembic Tutorial: alembic.sqlalchemy.org/en/latest/tutorial.html
- Pydantic Validators: docs.pydantic.dev/latest/concepts/validators
- Kode status HTTP: developer.mozilla.org/docs/Web/HTTP/Status